

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_3d536535-241\agent-a6182d4682a3137d2.jsonl`  
- SHA-256: `abccfb79992f2fd87c0ed38306c686b587f0f93f5c030d278c97e6ca0216e82c`  
- Source modified: `2026-06-16T03:20:03+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_3d536535-241`  
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T03:19:28.651Z)

Source Extractor

Research question: "Research question: **What concrete, recent (2022?2025) techniques and evaluation practices should a local, commercially-licensed badminton rally-detector adopt to close the quality gap with a strong cloud VLM (Gemini), given a tiny labeled set (~6 ? ~16 golden videos)?** Produce an adversarially-verified, cited report organized around the four dimensions below. Prefer specific named methods, papers (arXiv/venue), and reported numbers over generic advice; flag anything you cannot verify.

CONTEXT (so you target the real gap, not generic ML):

- System: a local pipeline detects the shuttle per-frame (WASB / TrackNet, MIT-licensed) ? a heuristic turns the trajectory into candidate rally windows ? today an optional Gemini "handoff" confirms rallies. We just measured a fully-local (no-AI) run vs Gemini on 3 matches: local emits far FEWER but MUCH LONGER windows (mean 65s vs 6.1s; one window spanned a whole 174s clip) ? it does NOT miss activity, it FAILS TO SEGMENT. Root cause (code-confirmed): a frame is "active" iff the shuttle is visible AND moving above a small per-second-normalized velocity threshold; active frames within a 2 s gap are merged; there is NO max-window cap and NO inactivity/boundary split. So "shuttle is moving" is conflated with "rally in progress," and dead-time + serves + knock-ups bridge rallies into mega-windows. Shuttle tracking itself is excellent (96?99% per-frame visibility).

- HARD CONSTRAINTS: weights/data/code must be MIT/Apache-2.0/BSD only (no viral/NC/custom licenses, reject Gemma-3/Llama-4-MAU-cap); we may NEVER train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs (terms/copyright), though paid LLMs may be used at inference; minors excluded from any training pool; runs on a single local GPU.

- WHAT WE ALREADY HAVE (do NOT re-survey the basics ? go deeper/newer): the architecture skeleton is known to be standard Temporal Action Localization/Segmentation (proposal+scorer) + late/score-level fusion + stacked generalization; in-domain prior art (MonoTrack, ShuttleSet, See et al. 2024/25 non-broadcast badminton rally start/end, Ghosh et al. point-segmentation, tennis/TT play-break state machines) is already cited. We have a working eval harness: leave-one-video-out (grouped by match), temporal-IoU F1 @0.5, F1@{0.1,0.25,0.5}, mAP@tIoU, segment-count-ratio, bootstrap 95% CIs, paired exact sign-test, Platt/isotonic calibration + Brier/ECE, a tiny numpy LogisticRegression scorer over per-window feature columns, an experiment leaderboard, and an "eval?serve" guard (a cue measured +0.074 F1 on permissive proposal windowing but ?0.008 on the coarser served windowing ? reverted). Available but default-OFF per-window signals: net-crossing count (rally gate), 5 person/court features, 3 audio-hit features.

THE FOUR DIMENSIONS:

1) WINDOWING & BOUNDARIES (Temporal Action Localization/Segmentation): beyond proposal+score ? what are the best *recent* methods for (a) turning a dense per-frame signal into well-segmented actions, (b) explicitly controlling OVER-segmentation/merging, (c) boundary detection/regression and splitting merged segments? Cover e.g. MS-TCN/MS-TCN++, ASRF (action-boundary regression), BMM/BSN boundary-matching, ActionFormer/TriDet/TemporalMaxer (actionness + boundary heads), smoothing/boundary losses, and the standard metrics for over-segmentation (segmental F1@k,

edit/Levenshtein score, mAP@tIoU). Which transfer to a 1-D shuttle-trajectory + auxiliary-cue setting on tiny data?

2) SHUTTLE-TRAJECTORY ? RALLY SIGNAL PROCESSING (racket-sports specific): how do published systems convert ball/shuttle tracking into rally/point segments and distinguish "in-play" from dead-time? Cover rally-state machines, serve/let detection, net-crossing & two-sided-occupancy cues, hit/contact (audio + kinematic) detection, and fps-invariant trajectory features. What features best separate "rally" from "shuttle merely moving"?

3) SMALL-DATA STRATEGIES: with ~6?16 labeled videos, what gives the most quality per label ? active learning (which videos/segments to label next; uncertainty/diversity/core-set), semi-/weakly-/point-supervised TAL (e.g. SF-Net, timestamp supervision), self-training/pseudo-labeling, and data-efficient TAL? Separately: the legally-clean ways to exploit a strong teacher (Gemini) at INFERENCE for weak/pseudo-labels or as an evaluation oracle WITHOUT its outputs becoming training data for owned weights ? what does the literature say about teacher-student / distillation boundaries and label-noise handling, and what governance distinctions matter?

4) EVALUATION METHODOLOGY & RIGOR at small n: trustworthy measurement with ~6?16 videos ? bootstrap/permutation tests, paired sign-tests, leave-one-group-out vs RL00CV bias, calibration (reliability, ECE), the over-segmentation-blindness of plain tIoU, shadow/A-B evaluation, regression gates, and concrete pitfalls (label noise, annotator agreement/IAA, multiple-comparison/over-fitting traps, eval-vs-serve drift). What metrics + protocols should a small-data sports-segmentation leaderboard standardize on?

Deliverable: a structured, cited report with, per dimension, the 3?5 highest-leverage techniques that APPLY to our constrained setting (small data, local, license-clean, 1-D trajectory + cues + learned scorer), each with: what it is, why it fits our over-merge/small-data problem, the expected difficulty/payoff, and the citation. Call out which ideas are low-effort high-confidence wins vs research bets. Be explicit about anything unverified or license-incompatible."

Fetch and extract key claims from this source:

****URL:**** <https://ai.google.dev/gemini-api/terms>

****Title:**** Gemini API Additional Terms of Service (Google AI for Developers)

****Found via:**** Knowledge-distillation licensing boundary + label-noise from teacher search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-06-16T03:19:33.148Z)

I'll fetch the Gemini API Terms of Service page to extract relevant claims about the teacher-student/distillation licensing boundary.

3. user (2026-06-16T03:19:43.700Z)

Gemini API Terms: Key Quotes on Model Development, Data Use, and Output Owners

****Effective Date:**** March 23, 2026 (Last updated April 28, 2026)

(1) Competing Model Development Restrictions

"You may not use the Services to develop models that compete with the Services (e.g., Gemini API or Google AI Studio)."

(2) Reverse Engineering & Component Extraction Prohibitions

"You also may not attempt to reverse engineer, extract or replicate any component of the Services, including the underlying data or models (e.g., parameter weights)."

(3) Generated Content Ownership

"Google won't claim ownership over that content. You acknowledge that Google may generate the same or similar content for others and that we reserve all rights to do so."

(4) Data Use for Unpaid Services

"Google uses the content you submit to the Services and any generated responses to provide, improve, and develop Google products and services and machine learning technologies."

(5) Data Use for Paid Services

"Google doesn't use your prompts (including associated system instructions, cached content, and files) or responses to improve our products."

(6) Attribution & Responsible Use

"You'll comply with applicable law in using generated content, which may require the provision of attribution to your users when returned as part of an API call."

4. assistant (2026-06-16T03:19:52.206Z)

I have the key content. This is a primary source (Google's official Terms of Service) directly bearing on Dimension 3 of the research question (legally-clean ways to exploit Gemini as a teacher without its outputs becoming training data).

5. user (2026-06-16T03:19:59.141Z)

Structured output provided successfully

6. assistant (2026-06-16T03:20:03.124Z)

Extracted the relevant claims from the Gemini API Terms of Service. This is a primary source (Google's official developer terms) directly governing Dimension 3 of the research question. Key findings: the Terms explicitly prohibit using the Services to develop competing models and to reverse-engineer/extract/replicate components including model weights, which is the contractual foundation for the project's hard constraint against training owned weights on Gemini outputs. A supporting paid-vs-unpaid data-use distinction and output-ownership clause are also relevant to using Gemini as an inference-time oracle.